Naming

- Names are used to uniquely identify *entities* in distributed systems
  - Entities may be processes, remote objects, etc.
- Names are mapped to entities' locations using name resolution

An entity can be identified by three types of references

1. Name
   - A name is a set of bits or characters that references an entity
   - Names can be human-friendly (or not)
     - E.g., DNS name www.google.com

2. Address
   - Every entity resides on an access point, and access point has an address
   - Reusable
     - E.g., IP Address + Port, phone number

3. Identifier
   - Identifiers are names that *uniquely* identify entities
   - A *true identifier* is a name with the following properties:
     - Each identifier refers to at most one entity
     - Each entity referred to by at most one identifier
     - An identifier always refers to the same entity (no identifier reuse)
   - E.g., MAC address

Naming Systems

- A *naming system* is simply a middleware that assists in name resolution
- Naming systems can be classified into three classes, based on the type of names used
  a. Flat naming
  b. Structured naming
  c. Attribute-based naming

Challenge

- A centralized table
  - Not scalable in a large network
- Many requests for name resolution
  - Often distributed across multiple machines

A. Flat Names

- String of random bits
- Does not contain any information on how to locate the access point of its associated entity

Types of name resolution mechanisms for flat names:

1. Table Lookup
2. Broadcasting
3. Forwarding pointers
4. Distributed Hash Tables (DHTs)

1. *Table Lookup*

- The most common implementation of a context is a table of {*name*, *value*} pairs
- When the implementation of a context is a table, the name-mapping algorithm is just a lookup of the name in that table
- The table itself may be complex, involving hashing or B- trees, but the basic idea is same
- Binding a new name to a value consists of adding that {*name*, *value*} pair to the table
    - E.g., Page Table

2. *Broadcasting*

Approach:

- Broadcast the name/address to the whole network
- The entity associated with the name responds with its current identifier
    - E.g., Address Resolution Protocol (ARP)
    - Resolve an IP address(address) to a MAC address(identifier)
    - A data link protocol that operates below the network layer

Challenges:

- Not scalable in large networks
    - This technique leads to flooding the network with broadcast messages
- Requires all entities to *listen* (or *snoop*) to all requests

3. *Forwarding pointers*

- Mobile entities
- When an entity moves, it leaves a pointer to where it went
    - For name resolution, follow the pointer chain
- A popular approach to locate mobile entities

Advantage:

- Dereferencing can be made transparent to client

Issues:

- Geographical scalability problems:
    - Chain can be very long for highly mobile entities
    - Long chains are not fault tolerant
    - High latency when dereferencing
- Need chain reduction mechanisms
    - Update client's reference when the most recent location is found

4. *Distributed Hash Table*

Scalable DHT lookup:

- Key/value store spread over millions of nodes
- Typical DHT interface:
    - put(key, value)

- get(key) -> value
- Weak consistency
  - Likely that get(k) sees put(k), but no guarantee

Challenges:

- Millions of participating nodes

Basic idea

- Impose a data structure (e.g., tree) over the nodes
  - Each node has references to only a few other nodes
- Lookups traverse the data structure "routing" (hop from node to node)
- DHT should route get() to same node as previous put()
- Example: The "Chord" peer-to-peer lookup system
  - Kademlia, the DHT used by BitTorrent, is inspired by Chord

Naïve approach:

- Query (get(key) or put(key, value)) is at some node
- Node needs to forward the query to a node "closer" to key
  - If we keep moving query closer, eventually we'll hit key's successor
- Each node knows its successor on the ring
  - I.e., forward query in a clockwise direction until done
- n.successor must be correct!
- Otherwise, we may skip over the responsible node and get(k) won't see data inserted by put(k)

Issue

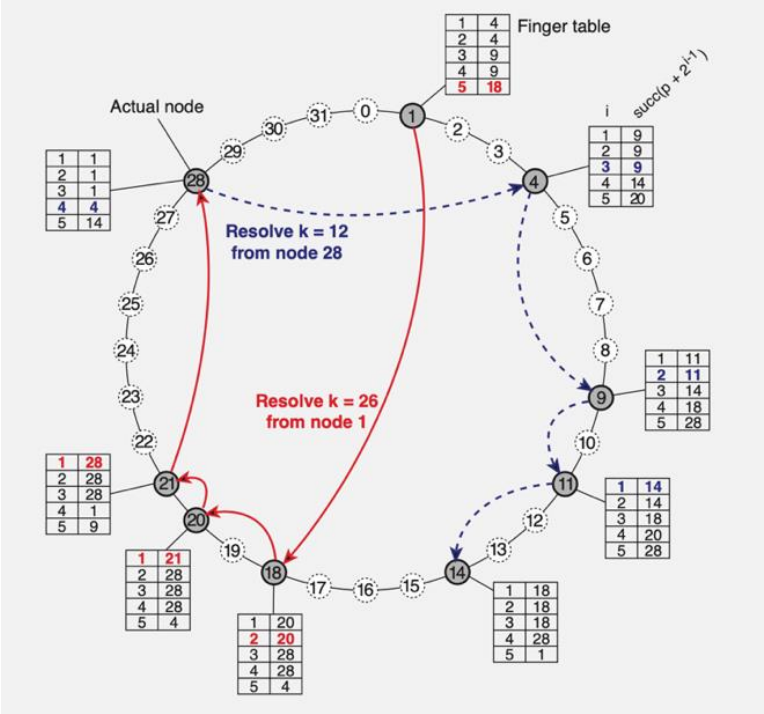
- Forwarding through successor is slow
- Data structure is a linked list: $O(n)$
- Can we make it more like a binary search?
  - Need to be able to halve the distance at each step

Faster Approach:

- $log(n)$ "finger table" routing
- Each node has an m-bit random identifier
- Each entity has an m-bit random key
- An entity with key k is located on a node with the smallest identifier that satisfies $id >= k$, denoted as $succ(k)$
- The task is key lookup i.e., to resolve an m-bit key to the address of $succ(k)$

Key Lookup in Chord

- Resolving key 26 from node 1 and key 12 from node *28*
- Here, m = 5, So number of nodes = 32



Why do lookups now take log(n) hops?

- One of the fingers must take you roughly half-way to target

Is $log(n)$ fast or slow?

- For a million nodes it's about 20 hops
- If each hop takes 50 ms, lookups take a second
- If each hop has 10% chance of failure, it's a couple of timeouts
  - So, good but not great
  - Though with complexity, you can get better real-time and reliability

Summary

- DHTs attractive for finding data in large p2p systems
- Decentralization seems good for high load, fault tolerance

Issues:

- $log(n)$ lookup time is not very fast
- The security problems are difficult
- *Churn* (nodes leaving and joining the network) is a problem, leads to incorrect routing tables, timeouts

B. Structured Naming

- Flat names are not convenient for humans to use
- Structured names are composed of simple, human-readable names,
  - E.g., file names(/home/userid/work/dist-systems/naming.txt), Internet domain names (www.cse.uta.edu)
- Structured names are often organized into a name space

Name Space

- A name space is a directed graph consisting of directory nodes and leaf nodes

1. Directory nodes
    - Directory node refers to other leaf or other directory nodes
    - The directory node stores a table where each outgoing edge is represented by (node identifier, edge label)
    - Each node can store any type of data i.e., state and/or address (e.g., to a different machine) and/or path
2. Leaf nodes
    - Each leaf node represents an entity
    - A leaf node generally stores the path to an entity (e.g., file systems)

Distributed Name Resolution

Distributed name resolution is responsible for mapping names to addresses in a system where:

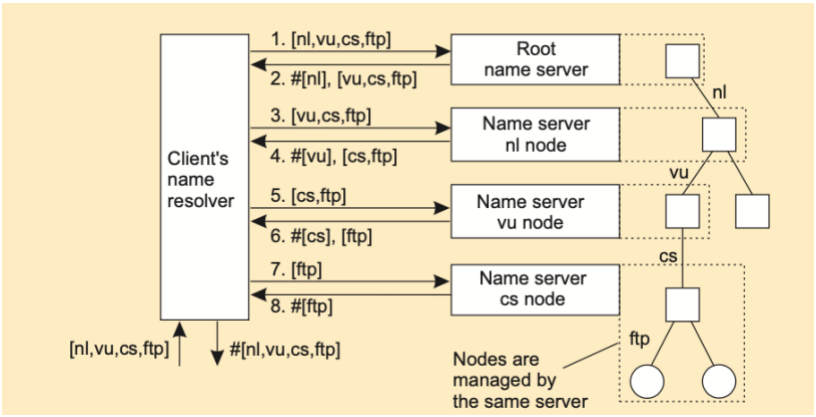
- Name servers are distributed among participating nodes
- Each name server has a local *name resolver*

Two major distributed name resolution algorithms:

1. Iterative Name Resolution
2. Recursive Name Resolution

Iterative name resolution

- Client hands over the complete name to *root name server*
- Root name server resolves the name as far as it can, and returns the result to the client
    - The root name server returns the address of the next-level name server (say, NLNS) if address is not completely resolved
- Client passes the unresolved part of the name to the NLNS
- NLNS resolves the name as far as it can, and returns the result to the client (and probably its next-level name server)
- The process continues until the full name is resolved

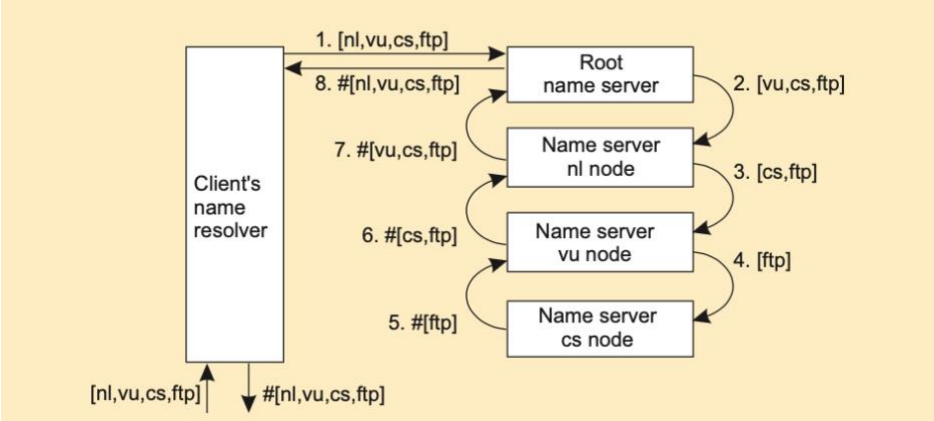


Recursive name resolution

- A path name can be thought of as a name that explicitly includes a reference to the context in which it should be resolved
- In some naming schemes, path names are written with the context reference first ((/usr/bin)/vim), in others with the context reference last (cse.(uta.edu))
- The recursive aspect of this description is that the explicit context reference is itself a path name that must be resolved
- So, we repeat the analysis as many times as needed until what was originally the most significant component of the path name is also the least significant component, at which point the resolver can do an ordinary table lookup using some context

DNS:

- Client provides the name to the root name server
- The root name server passes the result to the next name server it finds
- The process continues till the name is fully resolved



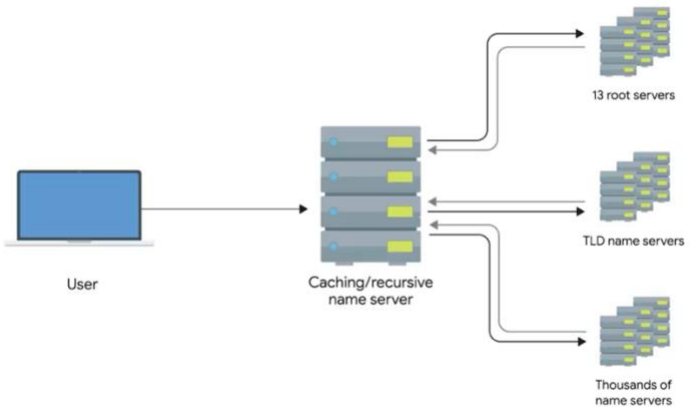
Recursive vs Iterative

- Recursive resolution demands more on each name server
- However, caching is more effective than iterative name resolution
  - Intermediate nodes can cache the result
- With iterative solution, only the client can cache

Domain Name System:

There are four primary types of DNS servers:

1. Caching name servers/recursive name servers
2. Root name servers
3. TLD name servers, and
4. Authoritative name servers



- A recursive name server performs name resolution of Fully Qualified Domain Name (FQDN)
- The first step is always to contact a root named server
- There are 13 total root name servers(?) and
    - They are responsible for directing queries toward the appropriate TLD (Top Level Domain)name server (handles .com, .org)
- In the past, these 13 root servers were distributed to very specific geographic regions,
    - But today, they're mostly distributed across the globe via anycast
- The root servers will respond to a DNS lookup with the TLD name server that should be queried
- TLD stands for top level domain and represents the top of the hierarchical DNS name resolution system
- A TLD is the last part of any domain name, using cse.uta.edu as an example again, the .edu portion should be thought of as the TLD
- For each TLD in existence, there is a TLD name server, but just like with root servers, this doesn't mean there's only physically one server in question, it's most likely a global distribution of anycast accessible servers responsible for each TLD
- The TLD name servers will respond again with a redirect, this time informing the computer performing the name lookup with what authoritative name server to contact
- Finally, the DNS lookup could be redirected at the authoritative server for cse.uta.edu which would finally provide the actual IP of the server in question
- Authoritative name servers are responsible for the last two parts of any domain name which is the resolution at which a single organization may be responsible for DNS lookups

What is Anycast?

- Anycast allows multiple devices or servers to share the same IP address
    - Different from unicast, in which each endpoint has its own, separate IP address
- Used to route traffic to different destinations depending on factors like location, congestion, or link health

- Provides multiple routing paths to a group of endpoints that are each assigned the same IP address
- Each device in the group advertises the same address on a network, and routing protocols are used to choose which is the best destination
- So, there aren't really only 13 physical route name servers anymore
    - 13 authorities that provide route name lookups as a service
- Using anycast, a computer can send a datagram to a specific IP but could see it routed to one of many different actual destinations depending on a few factors

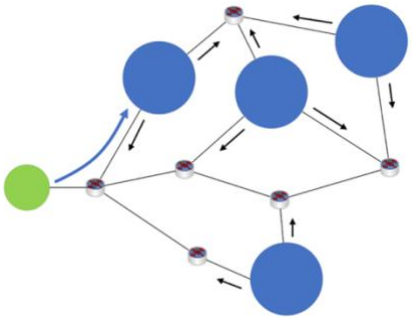


Figure: Four DNS servers located at different sites on a network each announce the same Anycast IP address (black arrows) to the network. A DNS client device sends out a request to the Anycast IP address. Network devices analyze the available routes and send the client's DNS query to the nearest location (blue arrow)

C. Attribute-based Naming

- As more information being made available, it becomes important to locate entities based on merely a description of that is needed

Attribute-based naming:

- Each entity is associated with a collection of attributes
- The naming system provides one of multiple entities that matches a user's description
- Attribute-based naming systems are often known as directory services
    - E.g., Lightweight Directory Access Protocol (LDAP)
- Each directory entry consists of (*attribute, value*) pairs, and is uniquely named to ease lookups

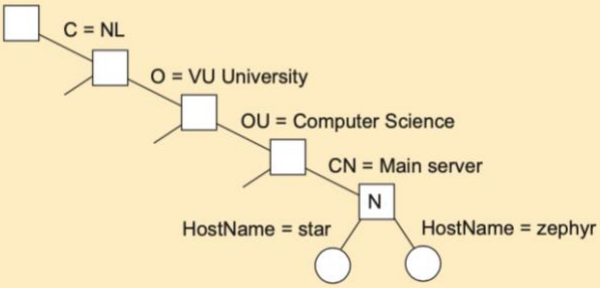
Attribute	Abbr.	Value
Country	*C*	NL
Locality	*L*	Amsterdam
Organization	*O*	VU University
OrganizationalUnit	*OU*	Computer Science
CommonName	*CN*	Main server
Mail_Servers	–	137.37.20.3, 130.37.24.6, 137.37.20.10
FTP_Server	–	130.37.20.20
WWW_Server	–	130.37.20.20

Directory Information Base:

- Collection of all directory entries in an LDAP service

Directory Information Tree:

- All the records in the DIB can be organized into a hierarchical tree called *Directory Information Tree (DIT)*
  - Each node represents a directory entry
- Part of a directory information tree



- Two directory entries matching the information:

Attribute	Value	Attribute	Value
Locality	*Amsterdam*	*Locality*	*Amsterdam*
Organization	*VU University*	*Organization*	*VU University*
OrganizationalUnit	*Computer Science*	*OrganizationalUnit*	*Computer Science*
CommonName	*Main server*	*CommonName*	*Main server*
HostName	*star*	*HostName*	*zephyr*
HostAddress	*192.31.231.42*	*HostAddress*	*137.37.20.10*

Result of `search(''(C=NL)(O=VU University)(OU=*)(CN=Main server)'')`

Optional Read:

- Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications by Ion Stoica, Robert Morris, David Karger, M. Frans Kaashoek, Hari Balakrishna
  - https://pdos.csail.mit.edu/papers/chord:sigcomm01/chord_sigcomm.pdf
- Kademlia: A Peer-to-peer Information System Based on the XOR Metric by Petar Maymounkov and David Mazieres
  - https://www.scs.stanford.edu/~dm/home/papers/kpos.pdf